บทที่ 4

เธรด

ในระบบปฏิบัติการสมัยใหม่ *เธรด (Thread)* ถือเป็นหน่วยย่อยของการทำงานภายในโพรเซส (process) ที่มีบทบาทสำคัญอย่างยิ่งในการเพิ่มประสิทธิภาพการทำงานของระบบคอมพิวเตอร์ เธรดช่วยให้โปรแกรมหนึ่งโปรแกรมสามารถแบ่งงานย่อยออกเป็นหลายสายการประมวลผลที่ทำงานพร้อมกัน (concurrently) ภายในโพรเซสเดียวกัน โดยแต่ละเธรดสามารถดำเนินงานได้อย่างอิสระ แต่ยังคงใช้ทรัพยากรร่วมกับเธรดอื่นในโพรเซสเดียวกัน เช่น หน่วยความจำและแฟ้มข้อมูล ทำให้ลดความซ้ำซ้อนในการจัดสรรทรัพยากรเมื่อเปรียบเทียบกับการใช้หลายโพรเซส ในบริบทของระบบคอมพิวเตอร์ที่ต้องการความเร็วในการประมวลผลหรือการตอบสนองแบบเรียลไทม์ เช่น แอปพลิเคชันด้านมัลติมีเดีย การสื่อสาร และระบบเซิร์ฟเวอร์ เป็นต้น เธรดจึงเป็นกลไกสำคัญที่ช่วยให้ระบบสามารถบริหารจัดการการทำงานแบบขนาน (parallelism) ได้อย่างมีประสิทธิภาพและยืดหยุ่นสูงสุด

4.1 เธรด

ในอดีตที่ระบบปฏิบัติการยังไม่รองรับแนวคิดของเธรด การประมวลผลภายในโพรเซสต้องดำเนินไปในลักษณะลำดับขั้น (sequential) โดยไม่สามารถแบ่งภาระงานออกเป็นหน่วยย่อยที่ทำงานพร้อมกันได้ ทำให้การทำงานที่ต้องตอบสนองหลายหน้าที่ เช่น การประมวลผล การป้อนข้อมูล และการสื่อสาร เกิดความล่าช้าและไม่มีประสิทธิภาพ หากต้องการให้ระบบสามารถทำงานหลายอย่างในเวลาเดียวกัน (concurrent processing) โปรแกรมจำเป็นต้องสร้างหลายโพรเซส ซึ่งแต่ละโพรเซสมีพื้นที่หน่วยความจำและทรัพยากรเป็นของตนเอง นำไปสู่ความสิ้นเปลืองทรัพยากรและความยุ่งยากในการสื่อสารระหว่างกัน ยิ่งไปกว่านั้น การสลับการทำงานระหว่างโพรเซส (process context switch) ซึ่งเป็นกระบวนการที่ระบบต้องจัดเก็บและโหลดสถานะของแต่ละโพรเซส ส่งผลให้เกิดค่าใช้จ่ายในการประมวลผลสูง และลดประสิทธิภาพของระบบลงอย่างมีนัยสำคัญ เธรดจึงได้รับการพัฒนาเพื่อแก้ไขปัญหาเหล่านี้ โดยอนุญาตให้โพรเซสหนึ่งสามารถมีหลายเธรดทำงานร่วมกันอย่างมีประสิทธิภาพ ภายใต้พื้นที่ทรัพยากรร่วมเดียวกัน ช่วยลดค่าใช้จ่ายในการสลับบริบท และเพิ่มความเร็วในการประมวลผลให้เหมาะสมกับระบบปฏิบัติการยุคใหม่

เธรด (Thread) คือ หน่วยย่อยของการทำงานภายในโพรเซส ซึ่งสามารถแบ่งปันทรัพยากรต่าง ๆ ของโพรเซสร่วมกันได้ เช่น พื้นที่หน่วยความจำ ตัวแปร หรือไฟล์ โดยในอดีต โพรเซสมักจะมีเพียงหนึ่งเธรด เรียกว่า **ซิงเกิลเธรด (single thread)** หรือ โพรเซสแบบหนัก (heavy weight process) ซึ่งเป็นรูปแบบที่พบในระบบปฏิบัติการรุ่นเก่า

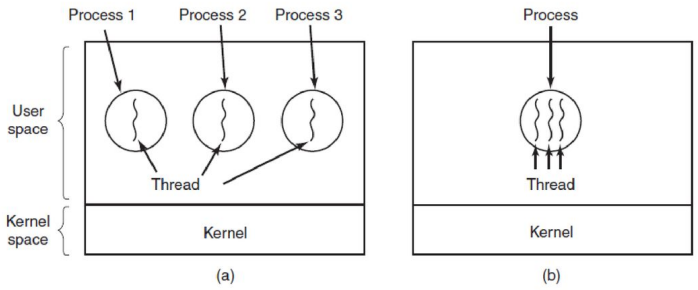
อย่างไรก็ตาม ระบบปฏิบัติการยุคใหม่มีการพัฒนาให้รองรับโพรเซสที่มีหลายเธรดภายในตัวเดียวกัน ซึ่งเรียกว่า **มัลติเธรด (Multithread)** หรือ โพรเซสแบบเบา (light weight process: LWP) โดยแต่ละเธรดสามารถทำงานต่างกันได้ แต่ยังอยู่ภายใต้โพรเซสเดียวกันและใช้ทรัพยากรร่วมกัน ทำให้การประมวลผล

มีประสิทธิภาพมากยิ่งขึ้น โดยเฉพาะในระบบที่มีหน่วยประมวลผลกลางหลายตัว (multiprocessor) หรือหลายแกนประมวลผลกลาง (multicore)

การใช้มัลติเธรด (multithreading) มีข้อดีหลายประการที่ส่งผลต่อประสิทธิภาพโดยรวมของโปรแกรมและระบบปฏิบัติการ โดยเฉพาะในด้านความเร็วในการประมวลผล และการตอบสนองต่อผู้ใช้ หนึ่งในข้อดีหลักคือการที่มัลติเธรดสามารถเพิ่มความเร็วในการทำงานของโปรแกรมได้อย่างชัดเจน เนื่องจากสามารถแบ่งงานออกเป็นหลายเธรดที่ทำงานได้พร้อมกันในลักษณะขนาน (parallel execution) หรือแบบพร้อมกัน (concurrent execution) ตัวอย่างเช่น ในโปรแกรมที่ต้องดาวน์โหลดข้อมูล ประมวลผล และแสดงผล เธรดแต่ละตัวสามารถรับผิดชอบหน้าที่เฉพาะของตนเองและทำงานไปพร้อมกันได้ ซึ่งจะลดเวลาการรอคอย (waiting time) และเพิ่มความลื่นไหลในการใช้งานของโปรแกรม

อีกประเด็นที่สำคัญคือข้อดีด้านการทำ context switch ในระบบปฏิบัติการแบบดั้งเดิมที่ใช้หลายโพรเซส (multi-process) การสลับการทำงานระหว่างโพรเซส (process context switch) เป็นกระบวนการที่ใช้ทรัพยากรสูง เพราะระบบต้องบันทึกและเรียกคืนข้อมูลทั้งหมดของโพรเซส เช่น ค่ารีจิสเตอร์ หน่วยความจำ และตารางไฟล์ ซึ่งก่อให้เกิดความล่าช้าในการประมวลผล อย่างไรก็ตาม ในระบบที่ใช้มัลติเธรด การสลับการทำงานระหว่างเธรด (thread context switch) มีความเบากว่าอย่างมาก เนื่องจากเธรดภายในโพรเซสเดียวกันใช้พื้นที่หน่วยความจำและทรัพยากรร่วมกัน การสลับบริบทจึงจำเป็นต้องเปลี่ยนแปลงเพียงข้อมูลส่วนเล็กน้อย เช่น ค่ารีจิสเตอร์ของแต่ละเธรด ไม่ต้องมีการโหลดทรัพยากรใหม่ทั้งหมดเหมือนกับการเปลี่ยนโพรเซส ส่งผลให้การบริหารจัดการเธรดมีความรวดเร็วและมีประสิทธิภาพมากกว่า ช่วยลด overhead ของระบบ และทำให้สามารถตอบสนองต่อผู้ใช้ได้อย่างฉับไว

รูปที่ 4.1 (a) แสดงให้เห็นระบบที่แต่ละโพรเซสควบคุมเพียงหนึ่งเธรด โดยมีพื้นที่หน่วยความจำของตนเองแยกจากกัน ในขณะที่รูปที่ 4.1 (b) แสดงโพรเซสที่ประกอบด้วยหลายเธรดซึ่งใช้พื้นที่หน่วยความจำร่วมกัน ซึ่งเป็นลักษณะที่พบได้ทั่วไปในระบบสมัยใหม่

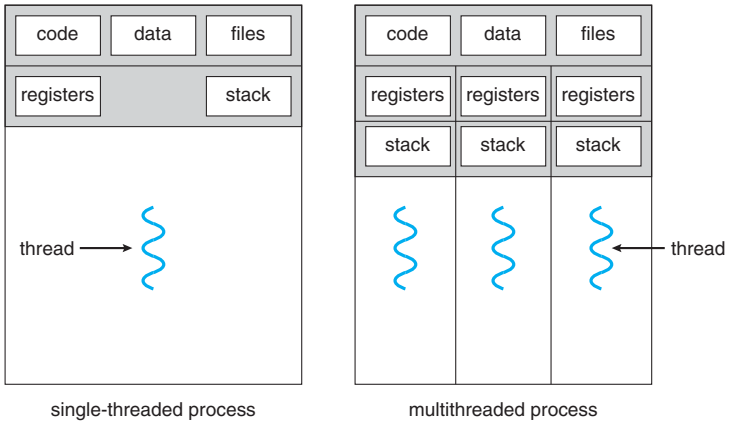


รูปที่ 4.1: (a) โพรเซสแต่ละโพรเซสมี 1 เธรด (b) หนึ่งโพรเซสที่มี 3 เธรด

เธรดเป็นหน่วยพื้นฐานในการจัดสรรการใช้ประโยชน์จากหน่วยประมวลผลกลาง ภายในโพรเซสหนึ่งจะประกอบด้วยเธรด ซึ่งสามารถทำงานได้อย่างมีประสิทธิภาพ โดยเธรดเหล่านี้จะใช้โค้ด ข้อมูล และทรัพยากรร่วมกัน เช่น ไฟล์และอุปกรณ์ต่าง ๆ เป็นต้น ในอดีต โพรเซสแบบดั้งเดิม หรือที่เรียกว่า heavy weight process จะมีเพียง 1 เธรด ทำให้สามารถประมวลผลงานได้เพียงอย่างเดียวในแต่ละช่วงเวลา แต่หากโพรเซสมีหลายเธรด (มัลติเธรด) ก็จะสามารถดำเนินงานหลายอย่างได้พร้อมกันภายในโพรเซสเดียวกัน เพิ่มประสิทธิภาพและความยืดหยุ่นในการประมวลผล รูปที่ 4.2 แสดงองค์ประกอบของโพรเซสที่เป็นแบบซิงเกิลเธรด และแบบมัลติเธรด เพื่อเปรียบเทียบโครงสร้างและลักษณะการทำงาน

องค์ประกอบที่สำคัญของเธรด มีดังนี้:

- **Thread ID** : หมายเลขที่ใช้ระบุแต่ละเธรดภายในโพรเซส เพื่อใช้ในการจัดการและแยกแยะเธรดแต่ละตัว



รูปที่ 4.2: โพรเซสแบบซิงเกิลเธรดและมัลติเธรด

- **Program Counter** : ตัวชี้ตำแหน่งของคำสั่งถัดไปที่เธรดจะดำเนินการในโปรแกรม
- **Registers** : หน่วยความจำขนาดเล็กที่ใช้เก็บค่าตัวแปรหรือตัวเลขระหว่างการทำงานของเธรด
- **Stack** : พื้นที่หน่วยความจำที่ใช้จัดเก็บข้อมูลระหว่างการเรียกใช้ฟังก์ชัน เช่น พารามิเตอร์ ตัวแปรภายใน และค่าที่ต้องคืนกลับ

ในปัจจุบัน ซอฟต์แวร์ที่พัฒนาเพื่อใช้งานบนเครื่องคอมพิวเตอร์สมัยใหม่มักถูกออกแบบให้รองรับการทำงานแบบมัลติเธรด ซึ่งหมายถึงภายในหนึ่งโพรเซสสามารถมีหลายเธรดทำงานร่วมกันได้อย่างเป็นอิสระ ตัวอย่างเช่น โปรแกรมเว็บบราวเซอร์อาจประกอบด้วยเธรดหนึ่งสำหรับแสดงผลภาพและข้อความ อีกเธรดหนึ่งทำหน้าที่ดึงข้อมูลจากอินเทอร์เน็ต ในขณะที่บางโปรแกรม เช่น โปรแกรมประมวลผลคำ อาจใช้หลายเธรดแยกหน้าที่กันไป ได้แก่:

- เธรดที่แสดงผลภาพกราฟิก
- เธรดที่รับอินพุตจากแป้นพิมพ์
- เธรดที่ตรวจสอบการสะกดคำและไวยากรณ์ในพื้นหลัง

การทำงานแบบนี้ช่วยให้โปรแกรมมีความลื่นไหลและสามารถตอบสนองต่อผู้ใช้ได้อย่างมีประสิทธิภาพมากยิ่งขึ้น โดยเฉพาะในกรณีที่ระบบต้องรองรับผู้ใช้งานจำนวนมาก เช่น เว็บเซิร์ฟเวอร์ที่ให้บริการข้อมูลแก่ลูกข่าย หากระบบทำงานในรูปแบบซิงเกิลเธรด ซึ่งสามารถรับคำขอได้เพียงครั้งละหนึ่งการเชื่อมต่อ ลูกข่ายที่เหลือจะต้องรอคิวเพื่อรับบริการ ส่งผลให้เกิดความล่าช้าในการตอบสนองและลดประสิทธิภาพของระบบลงอย่างชัดเจน

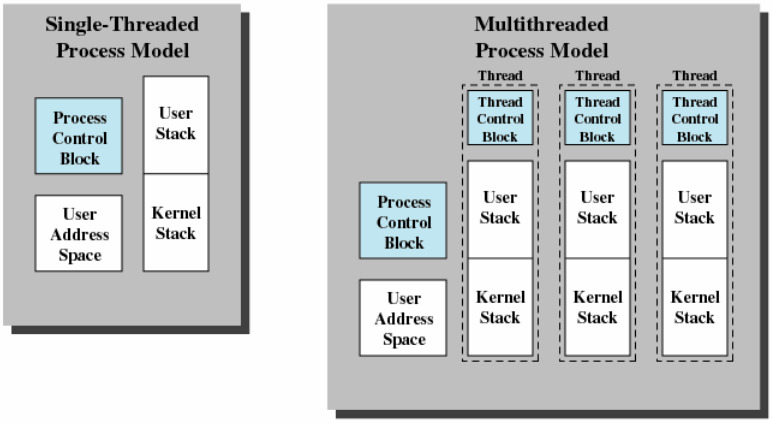
ในอดีต วิธีหนึ่งที่ใช้เพื่อรองรับการทำงานพร้อมกันคือ การให้เซิร์ฟเวอร์สร้างโพรเซสใหม่สำหรับแต่ละคำขอ (request) ที่เข้ามา โดยแต่ละโพรเซสจะมีหน้าที่รับผิดชอบเฉพาะคำขอนั้น ซึ่งสามารถทำงานแบบขนานได้ อย่างไรก็ตาม การสร้างโพรเซสใหม่ในแต่ละครั้งจำเป็นต้องใช้ทรัพยากรของระบบและเวลาในการจัดการสูง ไม่ว่าจะเป็นการจัดสรรหน่วยความจำ การตั้งค่าพื้นที่ข้อมูล และการโหลดข้อมูลของโพรเซสนั้น ๆ ส่งผลให้เกิดค่าใช้จ่ายในการประมวลผล (overhead) สูงและลดประสิทธิภาพโดยรวมของระบบลง เมื่อเปรียบเทียบกับการใช้เธรดที่มีน้ำหนักเบากว่าในการจัดการคำขอแต่ละรายการ

การพัฒนาระบบแบบมัลติเธรด จึงกลายเป็นแนวทางที่ได้รับความนิยมมากกว่า เนื่องจาก:

- การสร้างเธรดใหม่ใช้เวลาน้อยกว่าการสร้างโพรเซสใหม่
- เธรดภายในโพรเซสเดียวกันสามารถแชร์ข้อมูลและทรัพยากรร่วมกันได้สะดวก
- เพิ่มประสิทธิภาพการทำงานโดยรวมของระบบ โดยเฉพาะในงานที่ต้องตอบสนองต่อหลายคำขอพร้อมกัน

4.2 โพรเซสและเธรด

รูปที่ 4.3 แสดงให้เห็นถึงความแตกต่างระหว่างโครงสร้างแบบจำลองของโพรเซสที่มีเพียงเธรดเดียวกับโพรเซสที่มีหลายเธรด โดยเธรดนั้น จะแสดงได้ถึงหน่วยประมวลผลขนาดเล็กภายในโพรเซส ซึ่งสามารถทำงานได้อย่างมีประสิทธิภาพภายใต้การใช้ทรัพยากรร่วมกัน ไม่ว่าจะเป็นหน่วยความจำ คำสั่งโปรแกรม หรือแฟ้มข้อมูลต่าง ๆ การใช้หลายเธรดภายในโพรเซสเดียวช่วยให้ระบบสามารถแบ่งงานออกเป็นหลายส่วนที่ทำงานพร้อมกันได้ เพิ่มความเร็วและประสิทธิภาพโดยรวมของการประมวลผล



รูปที่ 4.3: ความแตกต่างระหว่างโพรเซสกับเธรด

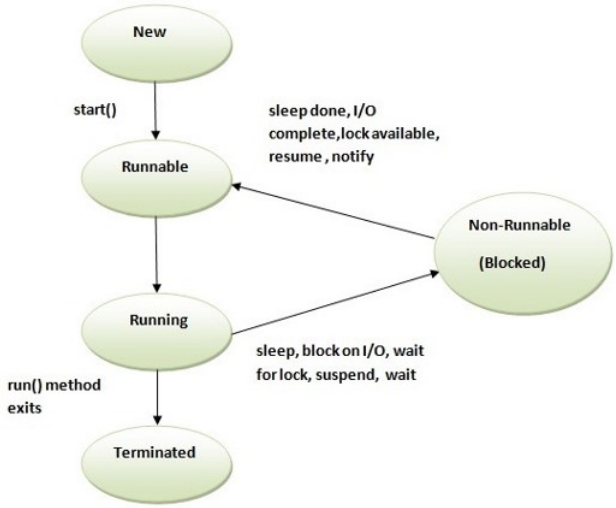
ในโพรเซสที่มีเพียงเธรดเดียว (single-threaded process model) จะมี **process control block (PCB)** เพียงหนึ่งชุด ซึ่งเป็นโครงสร้างที่ใช้เก็บข้อมูลสำคัญของโพรเซส เช่น พื้นที่หน่วยความจำ รีจีสเตอร์ สแตกของเคอร์เนล (kernel stack) และสแตกของผู้ใช้ (user stack) ทั้งหมดนี้ใช้สำหรับควบคุมและติดตามสถานะของโพรเซสในระหว่างการประมวลผล

ในทางกลับกัน สำหรับโพรเซสที่รองรับหลายเธรด (multithreaded process model) แม้จะมีเพียงหนึ่ง PCB เช่นเดียวกัน แต่ภายในโพรเซสนั้นจะมีเธรดหลายตัวทำงานร่วมกัน โดยแต่ละเธรดจะมี **thread control block (TCB)** ของตนเอง ซึ่งใช้เก็บข้อมูลจำเพาะของเธรด เช่น ค่ารีจีสเตอร์ของเธรด ลำดับความสำคัญ (priority) และสถานะของเธรด (state) รวมถึงมี user stack และ kernel stack แยกออกจากกันสำหรับแต่ละเธรด เพื่อรองรับการทำงานพร้อมกันอย่างมีประสิทธิภาพ

ทั้งนี้ เธรดทั้งหมดภายในโพรเซสเดียวกันจะใช้ทรัพยรร่วมกัน เช่น โค้ดโปรแกรม พื้นที่หน่วยความจำ และไฟล์ที่เปิดอยู่ ตัวอย่างเช่น หากเธรดใดเธรดหนึ่งเปิดไฟล์ขึ้นมา เธรดอื่น ๆ ที่อยู่ในโพรเซสเดียวกันก็สามารถเข้าถึงและใช้งานไฟล์นั้นร่วมกันได้ทันที โดยไม่จำเป็นต้องเปิดไฟล์ซ้ำอีกครั้ง ซึ่งถือเป็นข้อได้เปรียบสำคัญของการออกแบบระบบแบบมัลติเธรด

4.3 สถานะของเธรด

สถานะของเธรดมีความคล้ายคลึงกับโพรเซส ซึ่งสถานะของเธรด (*thread states*) แสดงถึงขั้นตอนต่าง ๆ ที่เธรดอยู่ระหว่างวงจรการทำงานของมัน โดยมีสถานะสำคัญ ได้แก่ *New*, *Runnable*, *Running*, *Blocked*, และ *Terminated* ดังแสดงในรูปที่ 4.4



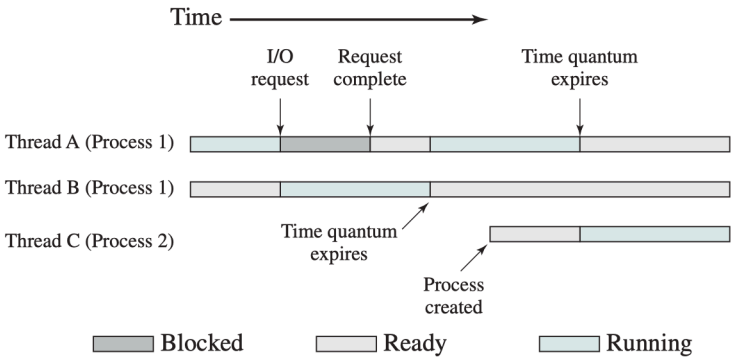
รูปที่ 4.4: สถานะของเธรด

สถานะต่าง ๆ ของเธรดสามารถอธิบายได้ดังนี้:

1. **New** – เป็นสถานะแรกเริ่มของเธรดเมื่อถูกสร้างขึ้น แต่ยังไม่ได้เริ่มทำงานในระบบ
2. **Runnable (หรือ Ready)** – เธรดอยู่ในสภาพพร้อมทำงาน และรอการจัดสรรหน่วยประมวลผลกลางจากระบบปฏิบัติการ
3. **Running** – เธรดได้รับหน่วยประมวลผลกลางแล้ว และกำลังอยู่ระหว่างการประมวลผลคำสั่ง
4. **Blocked** – เธรดอยู่ในสถานะรอการทำงานบางอย่าง เช่น รอข้อมูลจากดิสก์ หรือรอผลจากการสื่อสารกับอุปกรณ์ภายนอก ทำให้ไม่สามารถประมวลผลได้ในขณะนั้น
5. **Terminated** – เธรดสิ้นสุดการทำงานแล้ว ระบบจะทำการคืนทรัพยากรที่เธรดใช้กลับไปยังระบบปฏิบัติการ เพื่อให้สามารถนำไปใช้งานต่อได้

รูปที่ 4.5 แสดงลำดับการทำงานของระบบมัลติเธรด โดยประกอบด้วยเธรดจากสองโพรเซส ได้แก่ Thread A และ Thread B ซึ่งอยู่ภายใต้ Process 1 และ Thread C ที่อยู่ใน Process 2 ระบบปฏิบัติการจะสลับการทำงานของเธรดแต่ละตัวตามลำดับเวลาและสถานะของเธรด ได้แก่ *Running* (กำลังประมวลผล), *Ready* (รอรับหน่วยประมวลผลกลาง), และ *Blocked* (รอ I/O หรือทรัพยากรอื่น) ตัวอย่างเช่น Thread A เริ่มต้นด้วยการทำงาน แต่เมื่อมีการร้องขอ I/O ก็จะเข้าสู่สถานะ Blocked และเมื่อการร้องขอเสร็จสิ้นจึงกลับเข้าสู่สถานะ Ready และสามารถกลับมาทำงานใหม่ได้

การมี Thread B ภายใน Process 1 ช่วยให้โพรเซสสามารถทำงานต่อเนื่องได้แม้ว่า Thread A จะถูกระงับชั่วคราว โดยเมื่อ Thread A เข้าสู่สถานะ Blocked ระบบสามารถนำ Thread B ซึ่งอยู่ในสถานะ



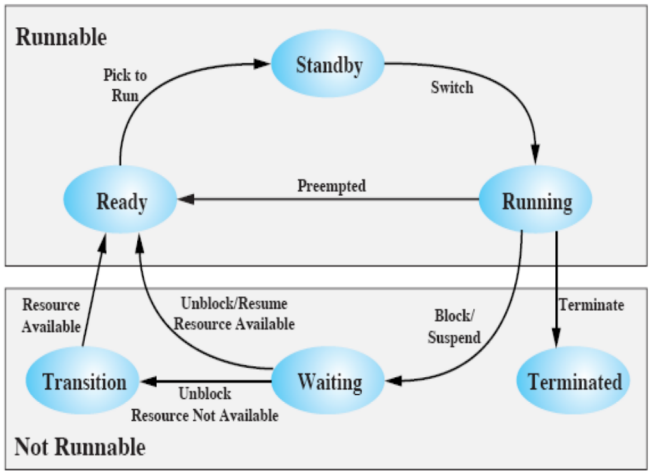
รูปที่ 4.5: ระบบมัลติเธรดที่มีหลายโพรเซสทำงานร่วมกัน

Ready มาทำงานต่อทันที ทำให้โพรเซสยังคงใช้หน่วยประมวลผลกลางอย่างต่อเนื่องโดยไม่ต้องรอ Thread A ส่งผลให้ลดเวลาที่หน่วยประมวลผลกลางอาจว่าง (idle time) และเพิ่มประสิทธิภาพในการใช้ทรัพยากรของระบบโดยรวม ซึ่งเป็นข้อได้เปรียบสำคัญของระบบที่รองรับการทำงานหลายเธรดภายในโพรเซสเดียว

4.4 เธรดในระบบปฏิบัติการวินโดว์

ในระบบปฏิบัติการวินโดวส์ (Windows) การจัดการเธรดถูกออกแบบให้อยู่ในรูปแบบของ *Object* เช่นเดียวกับโพรเซส โดยโพรเซสทำหน้าที่เป็นตัวแทนของโปรแกรมหรือแอปพลิเคชันที่ต้องการใช้งานทรัพยากรของระบบ เช่น หน่วยความจำ ไฟล์ หรืออุปกรณ์ต่าง ๆ ส่วนเธรดนั้นเป็นหน่วยย่อยภายในโพรเซส ซึ่งรับผิดชอบในการประมวลผลคำสั่งแบบลำดับ (sequential execution) การมีหลายเธรดภายในโพรเซสเดียวกันช่วยให้สามารถดำเนินงานหลายอย่างได้พร้อมกัน เพิ่มประสิทธิภาพการทำงานของระบบโดยรวม

ระบบวินโดวส์รองรับการขัดจังหวะ (interrupt) เธรด ซึ่งช่วยให้หน่วยประมวลผลกลาง สามารถสลับไปทำงานกับเธรดอื่นได้ตามลำดับความสำคัญ (priority) ที่กำหนดไว้ โดยสถานะต่าง ๆ ของเธรดจะแสดงไว้ในรูปที่ 4.6 ซึ่งสามารถอธิบายได้ดังนี้:



รูปที่ 4.6: การทำงานของ Windows Thread

1. **Ready** – เธรดอยู่ในสถานะพร้อมทำงาน และอยู่ในคิวเพื่อรอรับหน่วยประมวลผลกลาง โดยมีการจัดลำดับตามระดับความสำคัญ

2. **Standby** – เธรดได้รับการเลือกให้รันเป็นลำดับถัดไป และจะเปลี่ยนสถานะเป็น Running ทันทีเมื่อหน่วยประมวลผลกลางว่าง

3. **Running** – เธรดกำลังประมวลผลอยู่บนหน่วยประมวลผลกลางหากทำงานเสร็จ ถูกขัดจังหวะ หรือหมดโควต้าเวลา (time quantum) จะเปลี่ยนกลับไปสถานะ Ready

4. **Waiting** – เธรดถูกระงับการทำงานชั่วคราว เช่น รอทรัพยากรหรืออุปกรณ์ เมื่อเงื่อนไขที่รอสิ้นสุดจะกลับเข้าสู่กระบวนการเปลี่ยนสถานะเพื่อรอการประมวลผลอีกครั้ง

5. **Transition** – เธรดกำลังอยู่ในช่วงเปลี่ยนจาก Waiting กลับไปสู่ Ready แต่ยังไม่สามารถใช้งานได้ทันที เนื่องจากทรัพยากรที่จำเป็นยังไม่พร้อม

6. **Terminated** – เธรดสิ้นสุดการทำงาน ไม่ว่าจะจากการเสร็จสิ้นงาน การถูกยุติโดยเธรดอื่น หรือจากการสิ้นสุดของโพรเซสหลัก ทรัพยากรที่เธรดใช้จะถูกคืนกลับให้ระบบ

4.5 ข้อได้เปรียบของระบบมัลติเธรด

ระบบปฏิบัติการที่สนับสนุนการทำงานแบบมัลติเธรดมีข้อดีหลายประการ ทั้งในด้านประสิทธิภาพ ความยืดหยุ่น และการประหยัดทรัพยากร โดยสามารถสรุปข้อได้เปรียบหลัก ๆ ได้ดังนี้:

1. **ตอบสนองต่อผู้ใช้ได้ดี (responsiveness)** : โปรแกรมที่ทำงานแบบหลายเธรดสามารถตอบสนองต่อผู้ใช้ได้อย่างต่อเนื่อง แม้ว่าเธรดบางตัวอาจอยู่ในสถานะบล็อกหรือรอการประมวลผล ตัวอย่าง เช่น เว็บเบราว์เซอร์สามารถให้ผู้ใช้เลื่อนหน้าเว็บหรือพิมพ์ข้อความได้ แม้ขณะนั้นจะมีเธรดหนึ่งกำลังดาวน์โหลดภาพหรือข้อมูลจากเว็บไซต์อยู่

2. **ใช้ทรัพยากรร่วมกันได้อย่างมีประสิทธิภาพ (shared resources)** : เธรดที่อยู่ภายในโพรเซสเดียวกันสามารถเข้าถึงหน่วยความจำและทรัพยากรต่าง ๆ ร่วมกันได้ เช่น พื้นที่แอดเดรส หรือไฟล์ที่เปิดอยู่ ซึ่งช่วยลดความซับซ้อนในการสื่อสารระหว่างกัน และประหยัดการใช้หน่วยความจำเมื่อเทียบกับการใช้หลายโพรเซส

3. **ประหยัดทรัพยากรและลดค่าใช้จ่าย (economy)** : การสร้างเธรดใหม่ใช้ทรัพยากรน้อยกว่าการสร้างโพรเซสใหม่อย่างมาก เนื่องจากไม่ต้องจัดสรรหน่วยความจำแยกต่างหาก นอกจากนี้ การสลับการทำงาน (context switch) ระหว่างเธรดยังทำได้รวดเร็วกว่า ในระบบ Solaris 2 พบว่าการสร้างโพรเซสช้ากว่าการสร้างเธรดถึง 30 เท่า และ context switch ระหว่างโพรเซสก็ช้ากว่าเธรดถึง 5 เท่า

4. **รองรับการขยายระบบและการทำงานแบบขนาน (scalability)** : ในระบบที่มีหลายหน่วยประมวลผลหรือหลายคอร์ การใช้มัลติเธรดสามารถกระจายงานไปประมวลผลบนแต่ละคอร์พร้อมกันได้อย่างแท้จริง เพิ่มประสิทธิภาพของระบบโดยรวมได้อย่างมาก ขณะที่ระบบที่ใช้เธรดเดียวจะสามารถใช้ได้เพียงคอร์เดียวเท่านั้น แม้เครื่องจะมีหลายคอร์ก็ตาม ส่วนในระบบที่มีเพียงหน่วยประมวลผลกลางเดียว ระบบก็ยังสามารถสลับเธรดได้อย่างรวดเร็ว ทำให้ผู้ใช้รู้สึกว่าเกิดการทำงานพร้อมกัน (หimultaneity)

4.6 เธรดผู้ใช้งานและเธรดของระบบปฏิบัติการ

เธรดสามารถแบ่งออกได้เป็นสองประเภทหลักตามระดับการจัดการและการสนับสนุนจากระบบ ได้แก่ **เธรดผู้ใช้งาน (user thread)** และ**เธรดของระบบปฏิบัติการ (kernel thread)** ซึ่งทั้งสองประเภทมีลักษณะการทำงานและข้อดีข้อจำกัดที่แตกต่างกัน แต่สามารถนำมาใช้งานร่วมกันได้อย่างมีประสิทธิภาพ ทั้งนี้สามารถอธิบายรายละเอียดได้ดังนี้:

4.6.1 เธรดผู้ใช้งาน (User Thread)

เธรดผู้ใช้งาน คือ เธรดที่ถูกจัดการในระดับของผู้ใช้ โดยไม่จำเป็นต้องอาศัยการควบคุมจากระบบปฏิบัติการโดยตรง การสร้าง การจัดตารางเวลา (scheduling) และการควบคุมเธรดจะดำเนินการภายในไลบรารีของผู้ใช้ เช่น POSIX Pthreads หรือ Java Threads โดยไม่ต้องมีการเรียกใช้ system call ไปยังเคอร์เนล ส่งผลให้สามารถสร้างและจัดการเธรดได้รวดเร็ว ใช้ทรัพยากรน้อย และมี overhead ต่ำ

อย่างไรก็ตาม เธรดผู้ใช้งานมีข้อจำกัดสำคัญ โดยเฉพาะในระบบปฏิบัติการที่รองรับเพียงเธรดเดียวในแต่ละโพรเซส (single-threaded kernel) หากมีเธรดใดเธรดหนึ่งเรียกใช้ system call แล้วเข้าสู่สถานะบล็อก เธรดอื่น ๆ ภายในโพรเซสเดียวกัน แม้ว่าจะยังอยู่ในสถานะพร้อมทำงาน ก็จะไม่สามารถถูกนำไปประมวลผลได้ เนื่องจากระบบปฏิบัติการมองว่าโพรเซสนั้นทั้งหมดอยู่ในสถานะรอ

4.6.2 เธรดของระบบปฏิบัติการ (Kernel Thread)

เธรดของระบบปฏิบัติการ เป็นเธรดที่ได้รับการจัดการโดยตรงจากเคอร์เนล ซึ่งรับหน้าที่ในการสร้าง จัดเวลา และควบคุมการทำงานของเธรดในระดับของระบบปฏิบัติการ แม้ว่าการจัดการในลักษณะนี้จะมีค่าใช้จ่ายสูงกว่าการจัดการเธรดผู้ใช้งาน เนื่องจากต้องมีการติดต่อกับเคอร์เนลและใช้ทรัพยากรมากขึ้น แต่ก็มีความยืดหยุ่นและประสิทธิภาพสูงกว่าในหลายสถานการณ์

ตัวอย่างเช่น เมื่อเธรดหนึ่งถูกบล็อกจากการเรียก system call ระบบสามารถนำเธรดอื่นภายในโพรเซสเดียวกันมาทำงานต่อได้ทันที โดยไม่ต้องรอเธรดเดิมสิ้นสุดก่อน และในระบบที่มีหลายคอร์หรือหลายหน่วยประมวลผลกลาง เคอร์เนลยังสามารถกระจายเธรดต่าง ๆ ไปทำงานแบบขนานบนแต่ละคอร์ได้อย่างมีประสิทธิภาพ

ระบบปฏิบัติการที่สนับสนุนเธรดของระบบปฏิบัติการ ได้แก่ Windows, Linux, macOS และ Solaris ซึ่งล้วนรองรับการบริหารจัดการเธรดแบบ Kernel-level ได้อย่างยืดหยุ่น รองรับงานที่ซับซ้อน และสามารถขยายขนาดการประมวลผลให้สอดคล้องกับโครงสร้างของฮาร์ดแวร์ในยุคปัจจุบัน

4.7 รูปแบบของเธรด

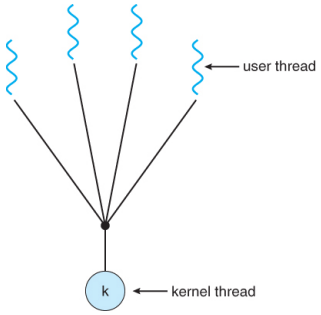
ระบบปฏิบัติการสมัยใหม่ส่วนใหญ่รองรับทั้ง **เธรดผู้ใช้งาน (user thread)** และ **เธรดของระบบปฏิบัติการ (kernel thread)** ซึ่งแต่ละระบบอาจใช้วิธีการจัดการเธรดแตกต่างกัน โดยเธรดผู้ใช้งานจะถูกควบคุมและจัดการภายในพื้นที่ของผู้ใช้ (user space) โดยไม่ต้องอาศัยการสนับสนุนจากเคอร์เนล ขณะที่เธรดของระบบปฏิบัติการจะถูกจัดการโดยตรงในระดับของเคอร์เนล (kernel space)

แม้ว่าเธรดทั้งสองประเภทจะมีลักษณะการทำงานและการควบคุมต่างกัน แต่ในทางปฏิบัติ ระบบจะต้องออกแบบให้ทั้งสองประเภทสามารถทำงานร่วมกันได้อย่างมีประสิทธิภาพ ดังนั้น ระบบปฏิบัติการจึงใช้วิธีการจับคู่หรือแมป (mapping) ระหว่างเธรดผู้ใช้งานกับเธรดของระบบปฏิบัติการในรูปแบบต่าง ๆ ซึ่งสามารถจำแนกได้เป็น 3 รูปแบบหลัก

4.7.1 รูปแบบ Many-to-One

ในรูปแบบนี้ เธรดผู้ใช้งานหลายตัวจะถูกแมปกับเธรดของระบบปฏิบัติการเพียงหนึ่งตัว ดังแสดงในรูปที่ 4.7 การจัดการเธรดทั้งหมดจะดำเนินการภายในไลบรารีของผู้ใช้ ทำให้การสร้างและควบคุมเธรดทำได้รวดเร็วและใช้ทรัพยากรน้อย

อย่างไรก็ตาม ข้อจำกัดสำคัญของรูปแบบนี้คือ เมื่อเธรดใดเธรดหนึ่งเรียกใช้ system call และถูกบล็อก เธรดทั้งหมดในโพรเซสจะไม่สามารถทำงานต่อได้ นอกจากนี้ ยังไม่สามารถใช้ประโยชน์จากการประมวลผลแบบขนาน (Parallel Processing) บนระบบที่มีหลายหน่วยประมวลผลกลางหรือหลายคอร์ได้ ตัวอย่างของระบบที่ใช้รูปแบบนี้ เช่น Green Threads ใน Solaris 2 หรือไลบรารีเธรดผู้ใช้งานที่ทำงานบนระบบที่ไม่รองรับ kernel thread

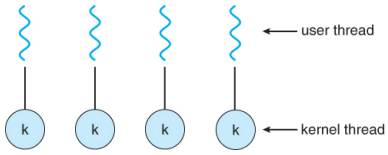


รูปที่ 4.7: รูปแบบ Many-to-One

4.7.2 รูปแบบ One-to-One

ในรูปแบบนี้ แต่ละเธรดผู้ใช้งานจะเชื่อมโยงกับเธรดของระบบปฏิบัติการหนึ่งตัวโดยตรง ดังแสดงในรูปที่ 4.8 รูปแบบนี้ช่วยให้ระบบสามารถประมวลผลเธรดหลายตัวได้พร้อมกันอย่างแท้จริง และสามารถหลีกเลี่ยงปัญหาการบล็อกของเธรด เช่น ถ้าเธรดหนึ่งถูกบล็อก เธรดอื่นยังสามารถทำงานต่อได้

อย่างไรก็ตาม การใช้รูปแบบนี้มีข้อจำกัดด้านทรัพยากร เนื่องจากการสร้างเธรดผู้ใช้งานแต่ละครั้งจะต้องสร้างเธรดของระบบปฏิบัติการร่วมด้วย ซึ่งอาจมีข้อจำกัดขึ้นอยู่กับระบบปฏิบัติการที่ใช้งาน ตัวอย่างของระบบที่ใช้รูปแบบนี้ เช่น Microsoft Windows



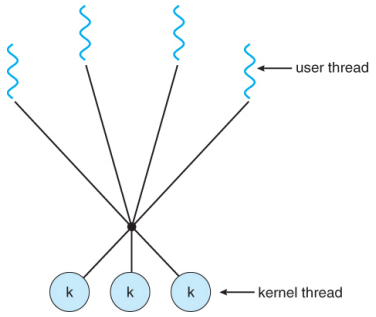
รูปที่ 4.8: รูปแบบ One-to-One

4.7.3 รูปแบบ Many-to-Many

รูปแบบนี้เป็นการผสมข้อดีของทั้งสองแบบก่อนหน้า โดยอนุญาตให้เธรดผู้ใช้งานหลายตัวสามารถแมปกับเธรดของระบบปฏิบัติการหลายตัวได้ ดังแสดงในรูปที่ 4.9 ซึ่งช่วยให้ผู้พัฒนาสามารถสร้างเธรดผู้ใช้งานได้

อย่างยืดหยุ่นตามต้องการ ขณะเดียวกัน ระบบปฏิบัติการสามารถควบคุมจำนวนเธรดของระบบให้เหมาะสมกับทรัพยากรที่มีอยู่ เพื่อเพิ่มประสิทธิภาพในการประมวลผลแบบขนาน

ข้อดีสำคัญของรูปแบบนี้คือ หากมีเธรดหนึ่งถูกบล็อกจากการเรียก system call ระบบสามารถนำเธรดอื่นในโพรเซสเดียวกันมาทำงานแทนได้ทันที ทำให้ระบบสามารถใช้ทรัพยากรได้อย่างมีประสิทธิภาพ โดยเฉพาะในสภาพแวดล้อมที่มีหลายหน่วยประมวลผล ตัวอย่างของระบบที่ใช้รูปแบบนี้ เช่น Solaris 2, IRIX, HP-UX และ Tru64 UNIX



รูปที่ 4.9: รูปแบบ Many-to-Many

4.8 การยกเลิกเธรด

การยกเลิกเธรด (thread cancellation) หมายถึง การหยุดการทำงานของเธรด ก่อนที่เธรดนั้นจะประมวลผลเสร็จสมบูรณ์ ซึ่งมักใช้ในกรณีที่เธรดหนึ่งทำงานสำเร็จแล้วและไม่มีความจำเป็นที่เธรดอื่น ๆ จะต้องทำงานต่อ ตัวอย่างเช่น ในโปรแกรมค้นหาข้อมูลที่มีการสร้างหลายเธรดเพื่อค้นหาผลลัพธ์จากฐานข้อมูล หากเธรดใดเธรดหนึ่งพบคำตอบแล้ว เธรดอื่น ๆ ที่เหลือสามารถถูกยกเลิกเพื่อประหยัดทรัพยากรได้ หรือในอีกกรณีหนึ่ง เช่น เมื่อผู้ใช้กดปุ่ม *Stop* บนเว็บบราวเซอร์ เธรดที่กำลังโหลดข้อมูลจากเครือข่ายจะถูกยกเลิกทันที

รูปแบบของการยกเลิกเธรดสามารถแบ่งออกได้เป็น 2 ประเภทหลัก ดังนี้:

1. **asynchronous cancellation** : เป็นการยกเลิกเธรดในทันที โดยไม่คำนึงว่าเธรดนั้นกำลังทำงานในส่วนใดของโปรแกรมอยู่ ซึ่งอาจส่งผลให้เกิดปัญหาด้านความปลอดภัยของข้อมูล โดยเฉพาะเมื่อเธรดกำลังเข้าถึงทรัพยากรที่ใช้ร่วมกับเธรดอื่น เช่น ไฟล์หรือหน่วยความจำ หากไม่มีการจัดการที่เหมาะสม อาจทำให้เกิดข้อมูลสูญหาย หรือระบบไม่เสถียร

2. **deferred cancellation** : เป็นการยกเลิกเธรดในลักษณะที่รอจังหวะ โดยเธรดเป้าหมายจะอนุญาตให้ยกเลิกได้เฉพาะในจุดที่ปลอดภัย (cancellation point) ซึ่งมักเป็นจุดที่ไม่ได้ใช้งานทรัพยากร่วมอยู่ วิธีนี้ช่วยให้สามารถจัดการทรัพยากรได้อย่างถูกต้อง และลดโอกาสเกิดข้อผิดพลาดจากการยกเลิกแบบไม่คาดคิด